PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
// Define the structure for a node
typedef struct Node {
    int data;
    struct Node *next;
}Node;
// Define the structure for a Circular Linked List
typedef struct {
    Node *head;
} CircularLinkedList;
// Initialize the Circular Linked List
void initCircularLinkedList(CircularLinkedList *list) {
    list->head = NULL;
}
 // Function to add an element to the Circular Linked List
void addCircular(CircularLinkedList *list, int element) {
    Node *new_node = (Node *)malloc(sizeof(Node));
    if (new_node == NULL) {
        fprintf(stderr, "Memory allocation failed\n");
        exit(EXIT_FAILURE);
    }
    new_node->data = element;
    new_node->next = NULL;
    if (list->head == NULL) {
        list->head = new_node;
        new_node->next = list->head;
    }
    else {
        Node *current = list->head;
        while (current->next != list->head) {
            current = current->next;
        }
        current->next = new_node;
        new_node->next = list->head;
    }
 }
// Function to remove an element from the Circular Linked List
void removeCircular(CircularLinkedList *list, int element) {
    if (list->head == NULL) {
        printf("List is empty\n");
        return;
    }
    Node *current = list->head;
    Node *prev = NULL;
    do {
        if (current->data == element) {
            if (current == list->head && current->next == list->head) {
                list->head = NULL;
```

```
            }
            else if (current == list->head) {
                Node *temp = list->head;
                while (temp->next != list->head) {
                    temp = temp->next;
                }
                temp->next = list->head->next;
                list->head = list->head->next;
            }
            else {
                prev->next = current->next;
            }
            free(current);
            printf("Element %d removed from the list\n", element);
            return;
        }
        prev = current;
        current = current->next;
    }
    while (current != list->head);
    printf("Element %d not found in the list\n", element);
}
// Function to display the Circular Linked List
void displayCircular(CircularLinkedList *list) {
    if (list->head == NULL) {
        printf("List is empty\n");
        return;
    }
    Node *current = list->head;
    printf("Circular Linked List: ");
    do {
        printf("%d ", current->data);
        current = current->next;
    } while (current != list->head);
    printf("\n");
}
// Function to get the size of the Circular Linked List
int sizeCircular(CircularLinkedList *list) {
    if (list->head == NULL) {
        return 0;
    }
    int count = 0;
    Node *current = list->head;
    do {
        count++;
        current = current->next;
    } while (current != list->head);
    return count;
}
int main() {
    CircularLinkedList list;
```

```
    initCircularLinkedList(&list);
    addCircular(&list, 1);
    addCircular(&list, 2);
    addCircular(&list, 3);
    printf("Circular Linked List after adding elements: ");
    displayCircular(&list);
    removeCircular(&list, 2);
    printf("Circular Linked List after removing element 2: ");
    displayCircular(&list);
    printf("Size of Circular Linked List: %d\n", sizeCircular(&list));
    return 0;
}
```